Group # 59
Names: Sabrina Wang, Maggie Huggins, Kyra McCarrick

Robot Design and Strategy Overview

We designed our robot to fit within the required 8 in × 8 in starting area while expanding to a maximum width of 12 inches during operation to maximize cube collection. The robot combined a passive mechanical expansion system with sensor-based navigation.

Mechanically, the robot used a cardboard outer shell mounted onto a two-wheel DC motor chassis. The shell consisted of a back wall and two side flaps that acted as expandable collection walls. Cardboard was chosen because it was lightweight, durable enough for competition use, and easy to prototype and modify quickly.

At the start of the match, the side flaps were folded inward at 90° angles so the robot remained within the starting size constraint. Two 130° brackets attached to the outer sides of the flaps limited their motion once released, creating a V-shaped opening approximately 12 inches wide. To hold the flaps closed before activation, removable standoffs were placed between the brackets and back wall. Strings connected these standoffs to the wheels. As the robot began moving, the wheels rotated forward and pulled the strings and removed the standoffs, allowing the single-piece cardboard shell to spring outward naturally until constrained by the 130° brackets.

The expanded V-shaped opening allowed the robot to passively funnel and retain cubes while driving forward. Because the robot never drove backwards, collected cubes remained contained within the side walls.

Electrically, the robot used one centrally mounted color sensor and two front-mounted QTI sensors connected to an Arduino. The color sensor detected transitions between the blue and yellow sections of the field, helping identify when the robot crossed the center of the board. The two QTI sensors were positioned at the front of the robot to detect the black field borders and prevent the robot from driving off the platform. All sensors and motors were connected to an Arduino, which processed sensor inputs and controlled the robot's movement.

During operation, the robot first drove forward to trigger the expansion mechanism. It continued moving until the color sensor detected a transition between the blue and yellow regions. The robot then turned 90° to the right and drove forward until the QTI sensors detected a black border. Upon reaching the border, the robot performed a 180° turn and continued driving toward the opposite side, repeating this back-and-forth motion for the remainder of the match. This strategy kept the robot traveling repeatedly through the center region of the field, increasing opportunities to collect cubes.

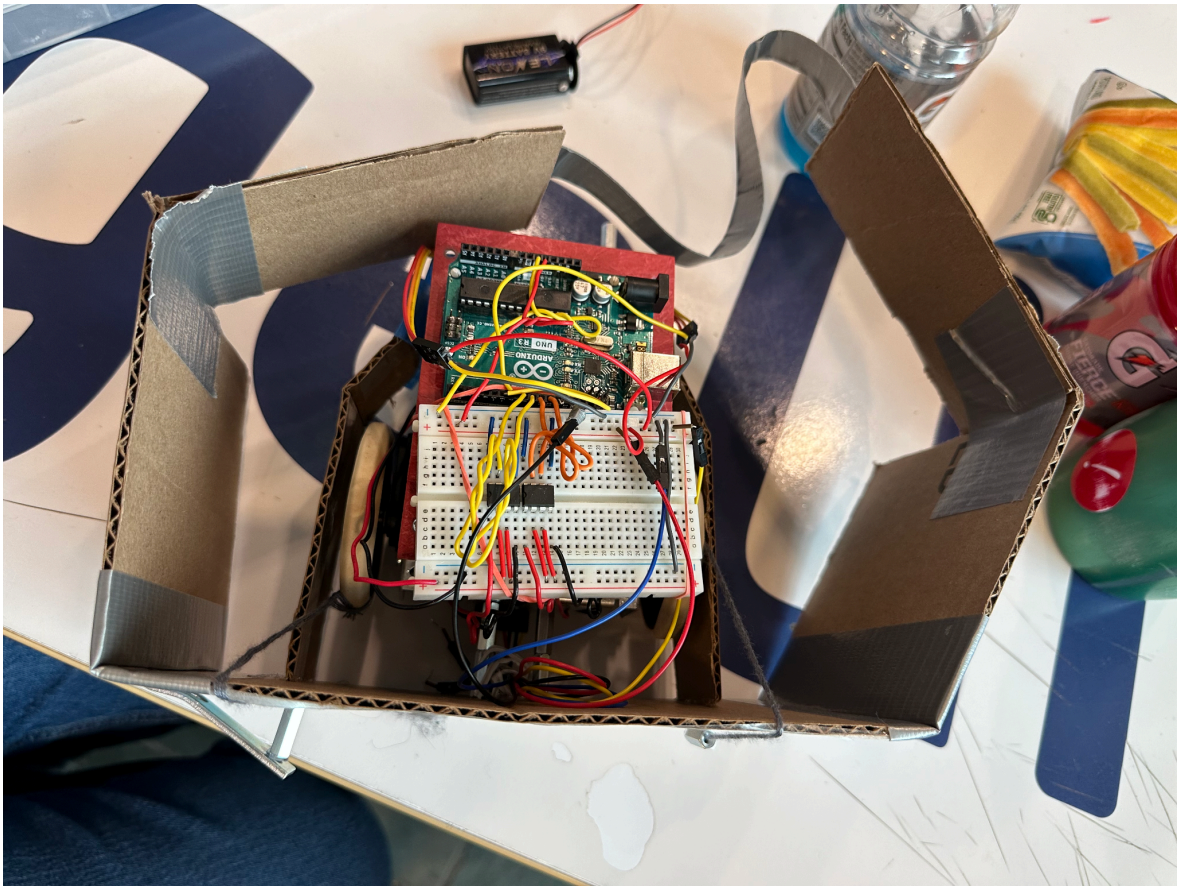


Figure 1. Final robot design (left side in starting position with locked standoff position and right side in expanded configuration with standoff released from starting position)

Design Process Reflection

The milestones helped us iterate through the basic robot design and functionality before developing our cube intake system. While working on the milestones, we attempted to operate without QTI sensors, but testing showed that the robot had difficulty detecting borders and would sometimes drift off the board. To solve this, we added two front-mounted QTI sensors, which improved reliability.

Our mechanical design also changed throughout the project. Initially, we built an expanding arm mechanism to collect blocks in front of the robot, but it did not fit within the 12-inch diameter constraint. We then pivoted to an expanding flap design. At first, we used polypropylene sheets, but they were too heavy, increased the risk of tipping, and did not spring back open as effectively as slitted cardboard. They were also more difficult to prototype and modify after cutting. Because of this, we switched to cardboard, which was lighter, easier to iterate on, and worked better for the passive flap system. Also, during testing, we found that collected cubes often jammed between the wheel and the wall of the robot, interfering with the robot's movement. To fix this, we added a wheel cage to keep cubes from colliding with the wheels.

Competition Analysis

Our robot performed well during the competition and was able to consistently collect 8-13 cubes. The navigation sensors worked reliably, especially the QTI sensors, which allowed the robot to consistently stay on the board throughout matches.

One unexpected issue we encountered was with our turning calibration. During testing the day before the competition, we found that the robot was under-turning, so we adjusted the right turn to achieve a 90-degree rotation. However, on competition day, the robot began over-turning, causing the right turn to exceed 90 degrees. We also found that when our robot collided with other robots, opponents would sometimes hit our side flaps, causing cubes to fall out of the robot's boundaries. It would have been beneficial to take fuller advantage of the allowed 8-inch length by extending the front of the robot with additional standoffs to keep other robots a safer distance from our side flaps. Additionally, our robot was slower than other robots and had a slower reaction time at the beginning of the competition after plugging in the battery.

Despite these issues, the robot's strengths were its reliable border detection and consistent expansion system. The expanding flaps worked effectively throughout the competition, allowing the robot to maintain a wide collection area and gather cubes efficiently.

Conclusions

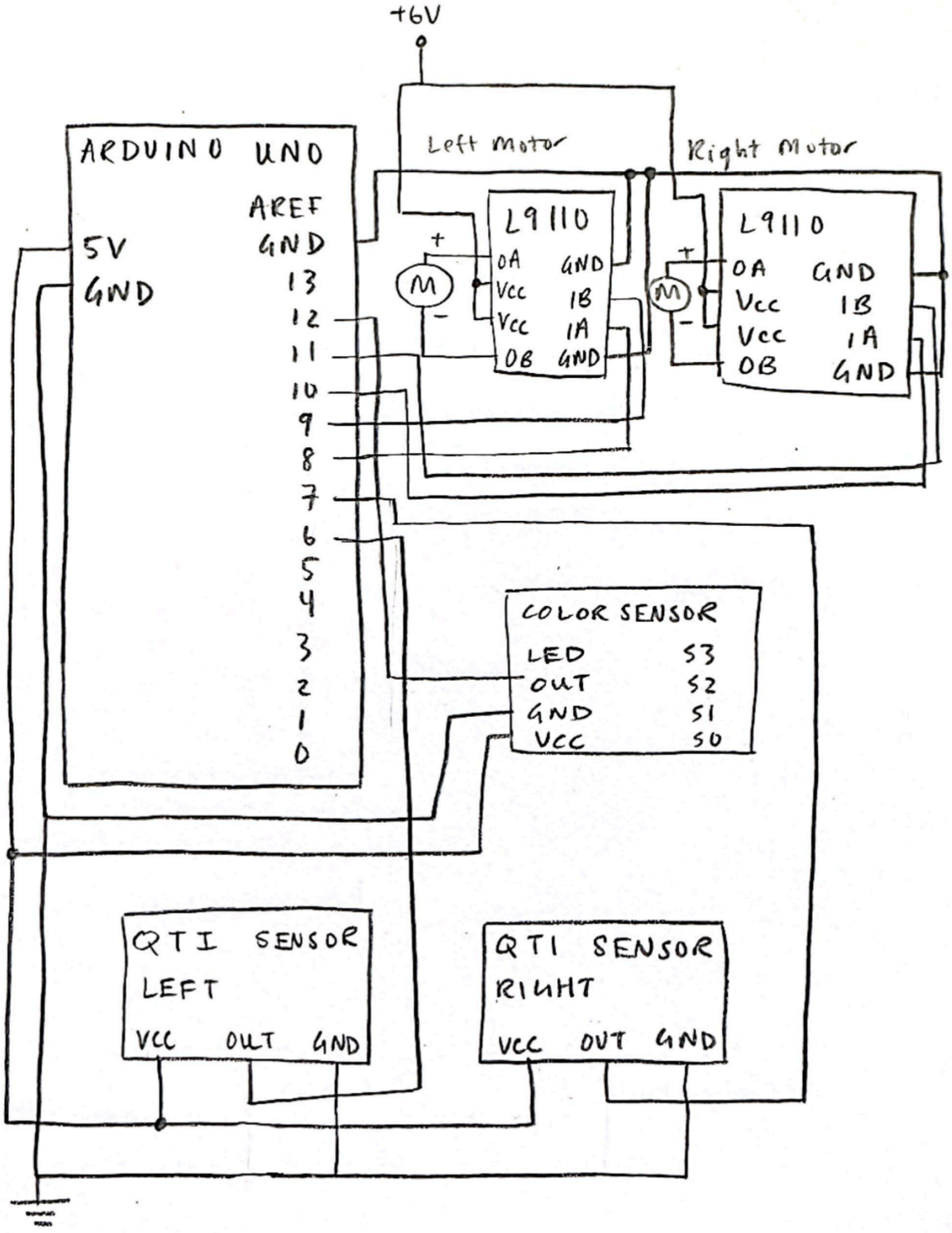
If we were to complete the project again with the same constraints and budget, we would focus more on optimizing details such as speed and reaction time. We would have liked to better understand why our robot had an initial delay and improve our own robot's startup response time. We also would have invested in larger wheels to increase the robot's overall speed.

For future students, we would recommend establishing the fundamentals of the robot first and foremost, especially reliable border detection and staying on the board consistently. Once the robot has dependable basic functionality and intake, additional improvements such as speed, reaction time, and collection efficiency can make a significant difference during competition. Small performance details became much more important during actual matches with other robots than when we practiced on the board alone in the lab.

Appendix A: Bill of Materials (BOM)

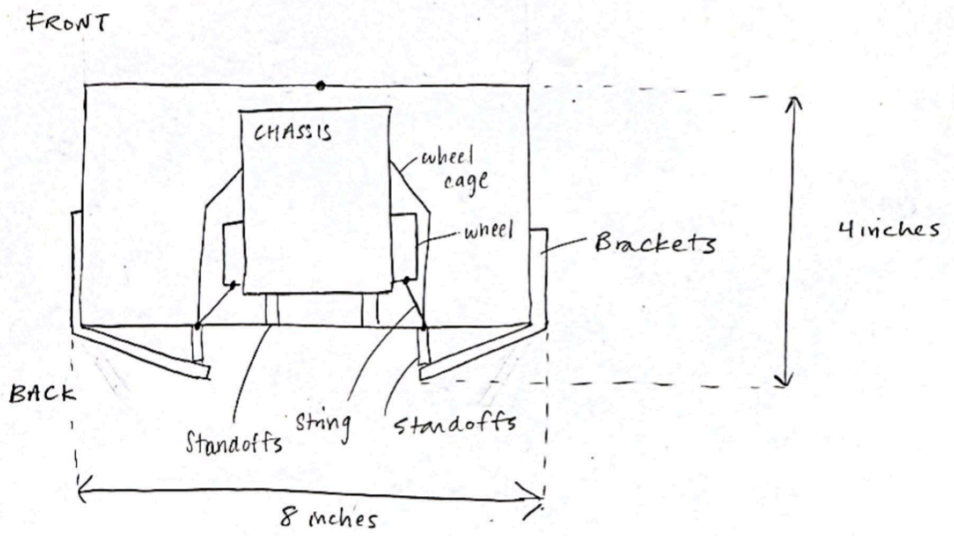
Item	Quantity	Unit Price	Total Price
Zinc Plated Steel Surface Mount Hinges with Holes	2	$10.75/pair	$10.75
Polypropylene Sheets	(1) 12”x24” sheet	$6.98/sheet	$6.98
Corner Bracket	2	$1.06	$2.12
Cardboard	(5) 36”x36” sheet	n/a	n/a
String	12”	n/a	n/a

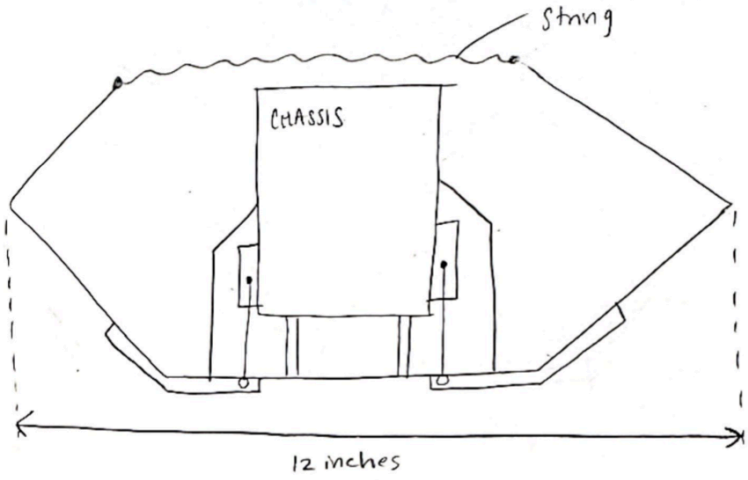
Appendix B: circuit diagram

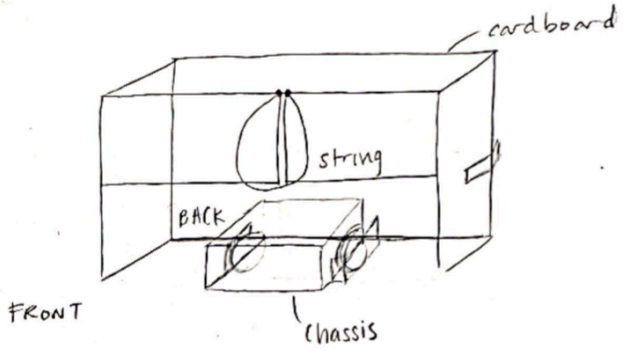


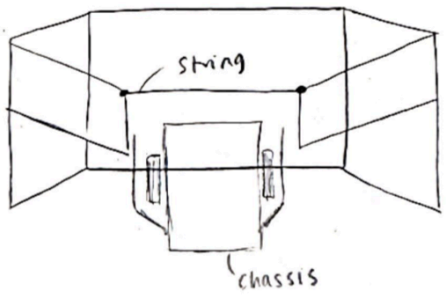
Appendix C: Drawings

Also include screenshots of part volumes and weight calculations (or photos of the parts being weighed) corresponding to the lines in your BOM.

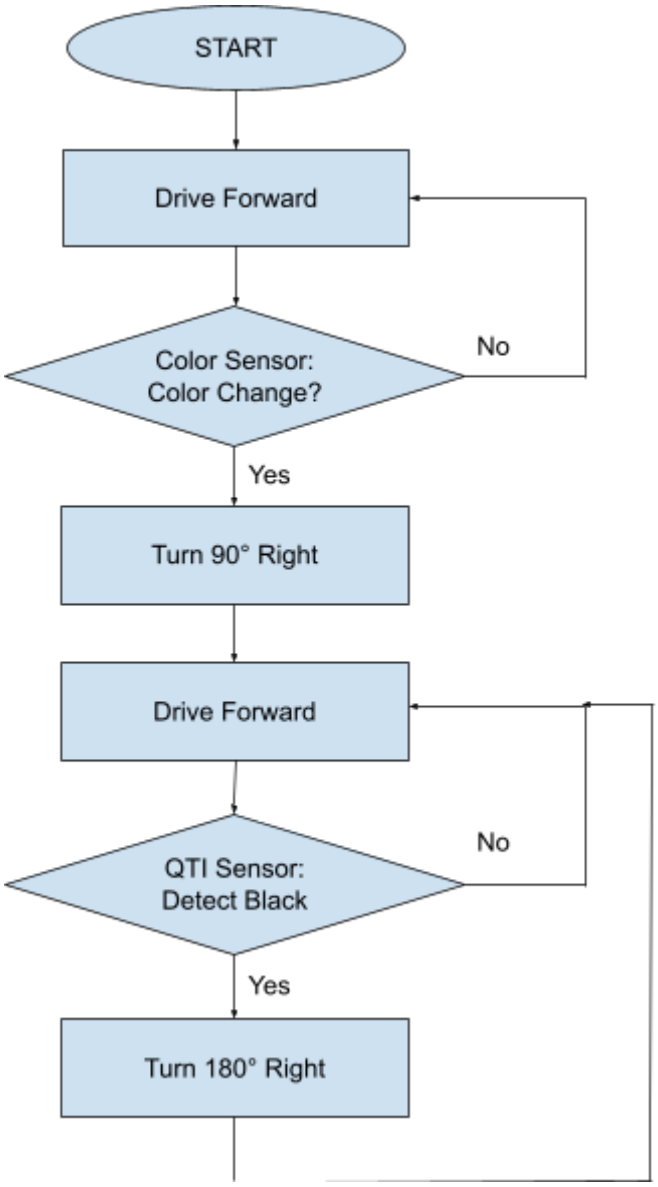








Appendix D: Flowchart



Appendix E: Code

```
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

volatile int timerValue = 0;

// Interrupt for Color Sensor on PB4 (Pin 12)
ISR(PCINT0_vect) {
    if (PINB & 0b00010000) {
        TCNT1 = 0; // Reset Timer1
    } else {
        timerValue = TCNT1; // Store period
    }
}

uint16_t readQTI(uint8_t pinMask) {
    // 1. Set pin as output and drive HIGH to charge the capacitor
    DDRD |= pinMask;
    PORTD |= pinMask;
    _delay_ms(1);

    // 2. Set pin as input, remove pull-up, and let capacitor discharge
    DDRD &= ~pinMask;
    PORTD &= ~pinMask;

    // 3. Count how long it takes for the pin to drop to logic LOW
    uint16_t count = 0;
    while (PIND & pinMask) {
        count++;
        if (count > 30000) break; // Timeout to prevent freezing if stuck
    }

    return count;
}

// Function to get pulse period in microseconds
int getColor() {
    PCMSK0 = 0b00010000; // Enable interrupt
    _delay_ms(10);       // Sample time
    PCMSK0 = 0b00000000; // Disable interrupt
    return (timerValue / 16);
}

// Movement functions
void forward() { PORTB = 0b00001001; }
void back()    { PORTB = 0b00000110; }
void right()   { PORTB = 0b00000101; }
void left()    { PORTB = 0b00001010; }
void stop()    { PORTB = 0b00000000; }

int main(void) {
    // UART Setup for 9600 baud
    UBRR0H = 0;
    UBRR0L = 103;
```

```
    UCSR0B = 0b00011000;
    UCSR0C = 0b00000110;

    DDRB = 0b00001111; // Pins 8-11 output, 12 input
    DDRD = 0b00001100; // Pins 2,3 output for filters

    // --- Timer 1 Setup (Color Sensor) ---
    TCCR1A = 0;
    TCCR1B = 0b00000001; // Prescaler = 1

    PCICR |= 0b00000001; // Enable PCINT0
    sei();               // Global interrupts

    // Determine Starting Color
    _delay_ms(500);
    int startVal = getColor();

    // --- PHASE 1: Drive to Center Transition ---
    while (1) {
        forward();
        int current = getColor();

        // Transition Logic
        if(abs(current - startVal) > 5){
            break;
        }
    }

    // --- PHASE 2: 90 Degree Right Turn ---
    stop();
    _delay_ms(500);
    right();
    _delay_ms(670);
    stop();
    _delay_ms(500);

    // --- PHASE 3: Infinite Back and Forth ---
    while (1) {

        uint16_t leftQTI = readQTI(0b01000000);
        uint16_t rightQTI = readQTI(0b10000000);

        // If black border is detected on either side
        if (leftQTI > 3000 || rightQTI > 3000) {
            stop();
            _delay_ms(200); // Brief pause before turning

            // Spin around 180 degrees (adjust 1340 based on your robot's
traction)
            right();
            _delay_ms(1675);

            stop();
            _delay_ms(200); // Settle before driving straight again
        } else {
            // Keep driving straight
            forward();
```

```
            }
        }
    }
```